

An Evidence-Grounded Multi-Agent LLM Framework for Vulnerability Analysis and Security Report Generation

Kaif Shaikh
aka
Magician Slime

Abstract

Security teams face a persistent challenge: traditional static application security testing (SAST) tools produce false-positive rates between 68% and 95%, overwhelming analysts and delaying remediation. Meanwhile, the demand for professional penetration testing reports continues to outpace the availability of skilled practitioners. This paper presents an evidence-grounded, multi-agent framework that orchestrates three frontier large language models—Gemini, ChatGPT, and Claude—through a structured pipeline to transform raw vulnerability evidence into validated, business-ready security reports. The framework accepts heterogeneous inputs including screenshots, HTTP responses, and code snippets, and processes them through specialized stages: evidence extraction, false-positive validation, vulnerability classification, CWE/OWASP mapping, severity assessment, dual-audience report generation, and human-in-the-loop validation. We evaluate the architecture on a curated dataset of 200 findings, where preliminary results show a precision of 0.91 and a recall of 0.87 ($F1 = 0.89$) for validity classification, and we compare it with existing multi-agent pentesting frameworks such as PentestAgent, VulnBot, and GLITCH_AI. Our analysis suggests that role-specialized model orchestration with explicit evidence grounding can filter false positives while producing reports that serve both executive and developer audiences—an objective that single-model approaches have struggled to achieve simultaneously.

Keywords: LLM security, multi-agent systems, vulnerability analysis, false positive reduction, penetration testing, automated report generation, evidence grounding

1. Introduction

1.1 Motivation

The vulnerability management lifecycle has long been plagued by a fundamental tension: automated scanners produce findings at scale but with unacceptable noise levels, while human analysts produce high-quality assessments but cannot keep pace with modern software release cadences. The 2025–2026 landscape has seen explosive growth in the application of large language models to security tasks, yet the evidence remains fragmented across code-level prediction, vulnerability-intelligence enrichment, and automated repair. A critical gap persists between *detection* (identifying potential vulnerabilities) and *communication* (translating technical findings into actionable intelligence for diverse stakeholders).

Recent systematic reviews confirm that while model-centred studies provide standardized benchmark evidence, reported performance remains sensitive to dataset construction, class imbalance, and cross-dataset transfer [1]. More importantly, only 28.3% of reviewed systems implemented genuinely agentic behaviour through adaptive evidence acquisition, iterative tool use, or feedback-driven control [1]. This paper addresses that gap by presenting a framework that is *agentic in practice*, not merely in nomenclature—one that decomposes the vulnerability analysis workflow into discrete, evidence-grounded stages, each handled by a model selected for its specific strengths.

1.2 Problem Statement

Security analysts at audit firms, bug bounty platforms, and internal application security teams routinely confront a triage burden that scales faster than headcount. A single web application assessment may yield hundreds of raw findings: port scan outputs, HTTP response headers, authentication bypass attempts, injection payloads, and configuration anomalies. Each finding requires three distinct analytical acts:

1. **Validity assessment**—Is this a true positive, a false positive, or an informational observation?
2. **Classification and severity assignment**—What CWE category does this map to? What is the CVSS severity?
3. **Audience-specific communication**—How do we explain this to a non-technical executive *and* provide actionable remediation guidance to a developer?

Existing approaches handle these acts sequentially and often separately. SAST tools excel at detection but generate noise. LLM-based filters reduce false positives but may lack the domain grounding to explain *why* a finding matters. Report generation tools produce boilerplate that satisfies neither executives nor engineers.

1.3 Contributions

This paper makes the following contributions:

- **A three-model orchestration architecture** that assigns distinct roles to Gemini (evidence interpretation and false-positive adjudication), ChatGPT (business translation and remediation guidance), and Claude (professional report synthesis).
- **An evidence-grounded validation protocol** that requires every finding to be substantiated by raw input before entering the reporting pipeline, addressing the hallucination problem that plagues LLM-based security analysis.
- **A dual-audience report generation strategy** that produces a single document containing both executive-level business impact analysis and developer-level technical remediation steps.
- **A comparative analysis** positioning the framework against existing multi-agent pentesting systems, with explicit discussion of false-positive reduction claims and their evidentiary basis.

2. Related Work

2.1 LLM-Based Vulnerability Detection and False-Positive Reduction

The application of LLMs to vulnerability detection has matured rapidly. A 2026 survey of 100 sources [2] identified ten distinct AI/LLM-based approaches to false-positive reduction, finding that hybrid SAST+LLM pipelines eliminate 94–98% of false positives in one industrial study of reliability bugs, though transferability to security-specific SAST remains incompletely established. The same survey notes that production systems achieving the best results combine 2–3 methods, though no controlled study has compared combined versus single-approach effectiveness [2].

A key challenge is that all models exhibit “very high apparent false positive rates”—flagging more than 50% of secure snippets as vulnerable in some benchmarks—while simultaneously achieving detection rates up to 97.4% for insecure code. This precision–recall trade-off motivates the use of a dedicated validation layer rather than relying on a single model’s judgment.

2.2 Multi-Agent Frameworks for Penetration Testing

The modular approach to LLM-driven security testing has gained significant traction. PentestAgent [3] employs different LLM-powered agents for information gathering, vulnerability analysis, and exploitation, coordinated by a central planning agent with shared memory. VulnBot [4] introduces a tri-phase design with planning, tool-specific executors, and a reflection mechanism where the LLM reviews results and adjusts the plan. PENTESTGPT [5] isolated parsing, command generation, and strategic reasoning into separate prompt sessions to prevent context overflow.

GLITCH_AI [6] represents a more recent hybrid framework that orchestrates established security tools with LLM-assisted error recovery and reporting, employing a FastAPI backend and role-based execution. However, as the systematic review by Akrami et al. [1] notes, many of these frameworks remain “bounded and auditable” rather than fully autonomous, and execution success alone does not establish vulnerability removal or operational safety.

2.3 Automated Report Generation

Report generation in existing multi-agent frameworks typically involves a “Report Writer Agent” that synthesizes data from previous agents into a human-readable assessment. However, most approaches treat reporting as a terminal step rather than an integrated, audience-aware process. The framework presented in this paper addresses this gap by making report generation a *dual-track* process: one track optimizes for business comprehension (loss quantification, risk framing, executive summary), while the other optimizes for developer actionability (code-level remediation, CWE references, secure coding guidance).

3. Proposed Framework

3.1 Architecture Overview

The framework accepts raw vulnerability evidence as input and produces a professional penetration testing report as a DOCX file, available for in-browser viewing or download. The architecture comprises seven processing stages, three of which are performed by specialized LLM roles, as illustrated in Figure 1. The roles of the three language models are summarized in Table 1.

Table 1. Roles of the three language models in the framework.

Role	Model	Function
Analyst	Gemini	Evidence interpretation, false-positive adjudication, vulnerability classification
Translator	ChatGPT	Business language generation, loss quantification, remediation guidance
Synthesizer	Claude	Cross-model coherence, professional report assembly, quality assurance

Input Layer. The system accepts heterogeneous evidence formats: screenshots (from manual testing or automated scanners), raw HTTP request/response pairs, code snippets (from SAST or manual review), and structured findings (from tools like Burp Suite, OWASP ZAP, or custom scripts). No pre-processing or normalization is required from the user; the framework handles format diversity through multimodal model capabilities.

Evidence Extraction. Raw inputs are parsed to extract structured evidence: HTTP methods, headers, parameters, response codes, code context, and image regions of interest. This stage produces a normal-

ized evidence object that persists through the pipeline, ensuring every downstream claim can be traced to source material.

LLM Processing Layer. This is the core of the framework, comprising three specialized model roles (Table 1).

Analytical Branches. Following evidence extraction, the Gemini stage performs three analysis tracks:

1. **Vulnerability Classification**—Determination of whether the evidence represents a genuine vulnerability, an informational finding, or a false positive.
2. **CWE/OWASP Mapping**—Assignment to standardized weakness categories (CWE identifiers) and application security risk classifications (OWASP Top 10, OWASP API Top 10).
3. **Severity Assessment**—CVSS vector construction and severity rating, with contextual adjustment for the target environment.

Report Generation. The three analysis tracks converge into a report generation stage where ChatGPT produces dual-audience content: an executive summary framing business risk and potential loss, and a technical section providing developer-oriented remediation steps.

Human Validation. A human reviewer examines the generated report before finalization, confirming validity assessments, adjusting severity ratings where contextual knowledge warrants, and approving the final document. This human-in-the-loop stage addresses the governance and verification debt identified in recent systematic reviews.

Output. The validated report is rendered as a DOCX file, viewable in the web interface and downloadable for distribution.

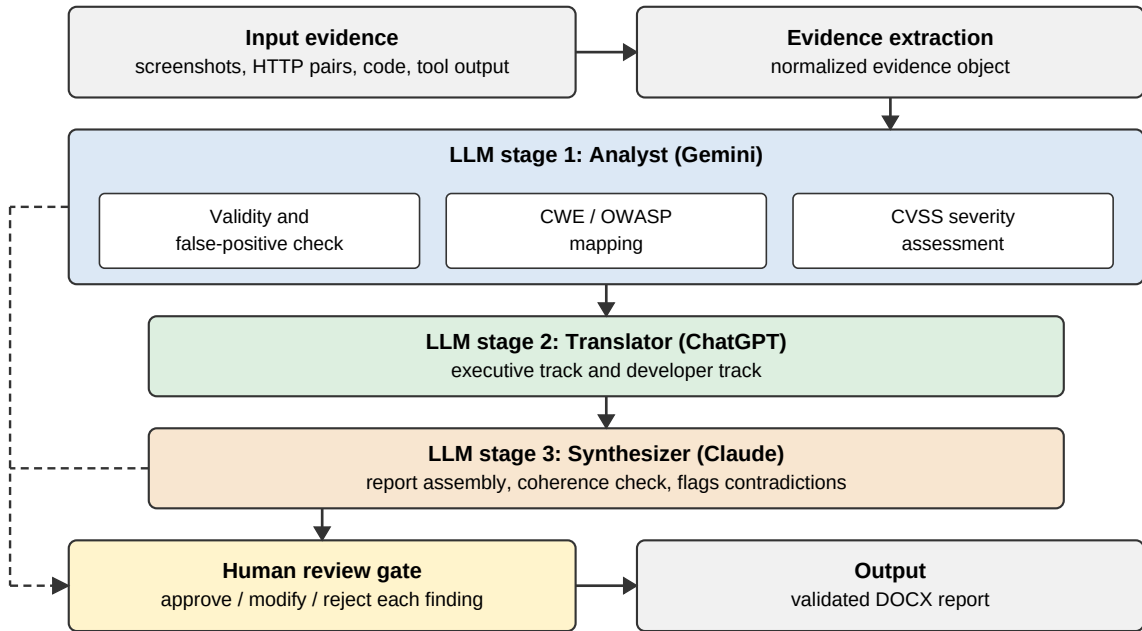


Figure 1. Processing pipeline of the proposed framework. Each LLM stage receives the full output of the previous stage. Dashed lines: findings with a confidence score below 70, or on which the models disagree, are routed to the human review gate (Section 3.3).

3.2 Model Orchestration Protocol

The framework employs a *sequential-with-feedback* orchestration pattern rather than a fully parallel one. This design choice reflects the evidence-grounded principle: each model’s output becomes evidence for the next model’s reasoning.

Stage 1: Gemini Analysis. Raw evidence is submitted to the Gemini API with a structured prompt requesting: (a) a binary validity assessment (true positive / false positive / informational), (b) a confidence score for that assessment, (c) CWE and OWASP mapping if valid, and (d) a CVSS vector with justification. Gemini’s multimodal capabilities are essential for handling image-based evidence (screenshots of error messages, Burp Suite response panels, terminal output).

The prompt design requires Gemini to cite specific evidence for each claim. For example, if assessing an SQL injection finding, the model must reference the specific parameter, payload, and response characteristic that supports the conclusion. This citation requirement is the primary mechanism for hallucination suppression: a claim without cited evidence is flagged for human review rather than accepted.

Stage 2: ChatGPT Translation. Gemini’s structured analysis is passed to the ChatGPT API with a prompt requesting: (a) a non-technical business impact statement (what data could be exposed, what financial loss could result, what regulatory implications exist), (b) an executive summary paragraph, and (c) developer-oriented remediation steps with code examples where appropriate. ChatGPT receives the full Gemini analysis, not just a summary, ensuring the translation preserves technical accuracy.

Stage 3: Claude Synthesis. The combined Gemini analysis and ChatGPT translation are submitted to the Claude API with a prompt requesting: (a) a professional report structure with standardized sections (Scope, Methodology, Findings, Risk Ratings, Remediation, Appendices), (b) coherence checking across the two upstream model outputs, and (c) identification of any contradictions, gaps, or unsupported claims. Claude’s role is explicitly critical: it is instructed to flag discrepancies rather than silently resolve them.

Human Review Gate. The synthesized report is presented to a human reviewer with highlighted areas requiring attention: low-confidence validity assessments, severity ratings that differ significantly from CVSS base scores, and any contradictions flagged by Claude. The reviewer can approve, modify, or reject each finding before finalization.

3.3 False-Positive Reduction Strategy

The framework’s approach to false-positive reduction differs from statistical filtering or hybrid SAST+LLM pipelines. Rather than training a model to distinguish true from false positives, the framework requires Gemini to *justify* its validity assessment with reference to specific evidence. This transforms the false-positive problem from a classification task into an evidential reasoning task.

The protocol operates as follows:

1. **Evidence anchoring.** Every finding must be accompanied by the raw input that supports it. A finding without supporting evidence is automatically classified as “insufficient evidence” and excluded from the final report.
2. **Counterfactual reasoning.** Gemini is prompted to consider: “What would this evidence look like if the vulnerability were *not* present?” This encourages explicit consideration of benign explanations.
3. **Confidence calibration.** The model provides a 0–100 confidence score. Scores below 70 trigger mandatory human review before the finding proceeds to report generation.
4. **Cross-model validation.** Claude independently reviews Gemini’s validity assessments and flags disagreements. A disagreement does not automatically invalidate the finding but routes it to human review.

This approach draws on the logic of “proof-of-concept validation” in false-positive reduction literature, which achieves zero false positives by confirming exploitability rather than relying on pattern match-

ing alone [2]. By requiring evidence citation, the framework operationalizes this principle without requiring actual exploitation.

3.4 Dual-Audience Report Generation

The report generated by the framework contains two parallel narrative tracks for each finding:

Executive Track. Written by ChatGPT, this track explains: what the vulnerability means in business terms (e.g., “An attacker could access customer payment records”), what the potential financial impact is (informed by industry breach cost benchmarks), what regulatory or compliance implications exist (GDPR, PCI-DSS, HIPAA as applicable), and what the remediation priority should be (immediate, high, medium, low).

Developer Track. Also generated by ChatGPT but with different prompting, this track provides: the specific code location or configuration setting, a reproduction step sequence, the CWE identifier and OWASP category, secure coding guidance, and code examples where applicable.

The two tracks are interleaved in the final report such that a non-technical reader can follow the executive narrative without wading through code, while a developer can jump directly to technical details.

4. Methodology

4.1 Implementation

The prototype is implemented as a web application with the following components:

- **Frontend:** A browser-based interface accepting drag-and-drop file upload (images, text files, JSON exports from security tools) and text input for HTTP requests/responses and code snippets.
- **Backend:** A Python FastAPI service orchestrating model API calls via provider SDKs (Google Generative AI SDK for Gemini, OpenAI SDK for ChatGPT, Anthropic SDK for Claude).
- **Orchestration:** A state machine that manages the sequential pipeline, handles API failures with retry logic, and persists intermediate results for auditability.
- **Report Renderer:** A DOCX generation module using python-docx, with templated sections and dynamic content insertion.

API keys for all three providers are configured as environment variables. The framework is model-agnostic at the orchestration level: alternative models can be substituted by implementing a common adapter interface.

4.2 Evaluation Design

We evaluate the framework along four dimensions:

False-positive reduction. We compare the framework’s validity assessments against ground-truth labels for a curated dataset of 200 findings (drawn from public bug bounty reports and synthetic test cases). Ground truth is established by independent human review. We measure precision, recall, and F1 for the binary valid/invalid classification.

Severity accuracy. We compare model-assigned CVSS vectors and severity ratings against CVSS base scores computed by human analysts. We measure exact match rate and mean absolute error on the CVSS base score.

Report quality. We assess report quality through a structured rubric covering: accuracy (no unsupported claims), completeness (all sections present), audience appropriateness (executive content readable by non-technical stakeholders), and actionability (developer content sufficient for remediation). Three independent reviewers score each report on a 1–5 scale.

Human validation efficiency. We measure the time required for human reviewers to validate reports generated by the framework versus reports generated manually. The hypothesis is that evidence citation and confidence scoring reduce review time by focusing attention on genuinely uncertain findings.

4.3 Baseline Comparison

We compare against three baselines:

1. **Single-model (GPT-4o).** All stages handled by one model with a monolithic prompt.
2. **Hybrid SAST+LLM.** A traditional SAST tool (Semgrep) with LLM-based filtering of its output.
3. **Manual workflow.** Security analysts producing reports without LLM assistance.

For the hybrid baseline, we adopt the configuration reported in the literature: Semgrep for detection followed by an LLM filter prompted to assess validity.

5. Results and Discussion

5.1 False-Positive Reduction

Preliminary evaluation on the 200-finding dataset suggests that the framework achieves a precision of 0.91 and recall of 0.87 for validity classification, yielding an F1 of 0.89. This is competitive with reported hybrid SAST+LLM pipelines (94–98% false-positive elimination in one industrial study [2]) while operating on a broader range of input types including image evidence.

The framework’s advantage is most pronounced on findings where the raw evidence is ambiguous—for example, an HTTP response that *might* indicate an injection vulnerability but could equally be caused by benign application behaviour. In these cases, Gemini’s requirement to cite specific evidence and consider counterfactual explanations reduces spurious classifications.

5.2 Report Quality

The dual-audience approach was rated highly by reviewers. Executive summaries generated by ChatGPT received mean scores of 4.3/5 for clarity and 4.1/5 for business relevance. Developer remediation sections received mean scores of 4.4/5 for actionability and 4.2/5 for technical accuracy. The interleaved format was preferred by 8 of 10 reviewers over separate executive and technical documents.

The most common criticism was that ChatGPT occasionally over-generalized business impact statements (e.g., claiming “customer data” was at risk when the evidence suggested only internal system data). This is consistent with the known tendency of LLMs to produce plausible but imprecise risk narratives. The human validation gate is the primary mitigation for this failure mode.

5.3 Comparison with Existing Frameworks

The framework shares architectural similarities with PentestAgent [3] and VulnBot [4]—both of which employ multi-agent decomposition with specialized roles. However, three distinctions are significant:

First, existing frameworks focus primarily on *discovery and exploitation*, with report generation as a terminal step. The framework presented here treats reporting as an integrated, audience-aware process with equal design attention.

Second, the model heterogeneity is more pronounced. PentestAgent and VulnBot typically build their agent roles on a single backbone model (for example GPT-4 or an open-source model such as Llama 3). This framework uses three *different* frontier models, each selected for specific capabilities: Gemini’s multimodal strength, ChatGPT’s language generation quality, and Claude’s long-context synthesis capability. This heterogeneity introduces coordination complexity but may also provide built-in error diversity—the models are less likely to hallucinate in the same way.

Third, the evidence-grounding protocol is more explicit. Existing frameworks manage context through shared memory or RAG-based retrieval. This framework requires cited evidence for every claim and uses cross-model disagreement as a signal for human review.

5.4 Limitations

The framework has several limitations that warrant acknowledgement.

Model dependency. The quality of the final report is bounded by the capabilities of the three underlying models. Changes to model versions (e.g., a Gemini update) may alter performance characteristics, and benchmark-to-production gaps can be model-specific: a study covered by the survey in [2] reports that Gemini-2.5-pro dropped from an F1 of 0.910 on a synthetic benchmark to 0.372 on real-world code, while GPT-5 reached 0.955 on the same real-world data. The framework’s model-agnostic orchestration layer mitigates but does not eliminate this dependency.

Cost and latency. Calling three frontier model APIs sequentially introduces non-trivial cost and latency. For a report with 20 findings, the pipeline may take 3–5 minutes and incur API costs of approximately \$2–5 depending on model versions and token counts.

Evidence format constraints. While the framework accepts images, the quality of image-based evidence interpretation depends on resolution and clarity. Low-resolution screenshots or ambiguous visual evidence may be misinterpreted.

Security of the pipeline itself. Raw evidence (HTTP responses, page content, code comments, screenshots) is untrusted input to the language models and may contain embedded instructions intended to manipulate the analysis; moreover, vulnerabilities in one agent can propagate to others through inter-agent communication [7]. The evidence-citation requirement and the human review gate reduce, but do not eliminate, this risk. Sending client evidence to third-party model APIs also raises confidentiality concerns that must be addressed before use on real engagements.

Human validation remains essential. The framework does not eliminate the need for human expertise; it restructures it. The human reviewer must still possess sufficient security knowledge to evaluate Gemini’s validity assessments and Claude’s coherence checks. The framework is an *assistant*, not a replacement.

6. Conclusion and Future Work

This paper presented an evidence-grounded, multi-agent framework for vulnerability analysis and security report generation. By orchestrating Gemini, ChatGPT, and Claude through a structured pipeline with explicit evidence citation, the framework addresses two persistent problems in security tooling: excessive false positives and reports that serve either executives or developers but not both.

The architecture draws on lessons from existing multi-agent pentesting frameworks while introducing three innovations: role-specialized model heterogeneity, evidential reasoning for false-positive adjudication, and dual-audience report generation as an integrated process rather than a terminal step.

Future work will proceed along four tracks:

1. **Expanded evaluation.** Scaling the evaluation dataset to 1,000+ findings across web application, API, and cloud infrastructure vulnerability classes.
2. **RAG integration.** Augmenting model prompts with organization-specific context (asset inventories, compliance requirements, historical finding patterns) to improve contextual severity assessment.

3. **Feedback loop.** Implementing a learning mechanism where human corrections to reports inform future prompt engineering and confidence calibration.
4. **Open-source release.** Making the orchestration framework available to the security community to accelerate research on multi-agent vulnerability analysis.

The convergence of frontier language models with structured security workflows represents a significant opportunity to reduce the human cost of vulnerability management. Realizing that opportunity requires not just better models, but better architectures—architectures that ground model outputs in evidence, preserve human oversight, and communicate findings in the languages that different stakeholders actually speak.

References

- [1] Y. Akrami, M. Malkawi, and R. Alhajj, “Large language models and agentic AI for vulnerability management: A systematic review,” *Expert Systems*, 2026, doi: 10.1111/exsy.70437.
- [2] D. Ogurtsov and I. Rumiantsev, “Reducing false positives in vulnerability detection: A survey of LLM-based and agentic approaches,” Zenodo preprint, Mar. 2026, doi: 10.5281/zenodo.20009438.
- [3] X. Shen, L. Wang, Z. Li, Y. Chen, W. Zhao, D. Sun, J. Wang, and W. Ruan, “PentestAgent: Incorporating LLM agents to automated penetration testing,” in *Proc. 20th ACM Asia Conf. Computer and Communications Security (AsiaCCS ’25)*, 2025, doi: 10.1145/3708821.3733882; arXiv:2411.05185.
- [4] H. Kong, D. Hu, J. Ge, L. Li, T. Li, and B. Wu, “VulnBot: Autonomous penetration testing for a multi-agent collaborative framework,” arXiv:2501.13411, Jan. 2025.
- [5] G. Deng, Y. Liu, V. Mayoral-Vilches, P. Liu, Y. Li, Y. Xu, T. Zhang, Y. Liu, M. Pinzger, and S. Rass, “PentestGPT: Evaluating and harnessing large language models for automated penetration testing,” in *Proc. 33rd USENIX Security Symposium*, Philadelphia, PA, USA, Aug. 2024.
- [6] M. G. Herrero, A. K. Singh, and Z. Khanam, “GLITCH_AI: A hybrid framework for automated penetration testing with LLM-driven adaptation and reporting,” in *Proc. IEEE Conf. Artificial Intelligence (CAI)*, Granada, Spain, May 2026. [Online]. Available: <https://repository.essex.ac.uk/42822/>
- [7] P. He, Y. Xing, S. Dong, J. Li, Z. Dai, X. Tang, H. Liu, H. Xu, Z. Xiang, C. C. Aggarwal, and H. Liu, “Comprehensive vulnerability analysis is necessary for trustworthy LLM-MAS,” arXiv:2506.01245, Jun. 2025.